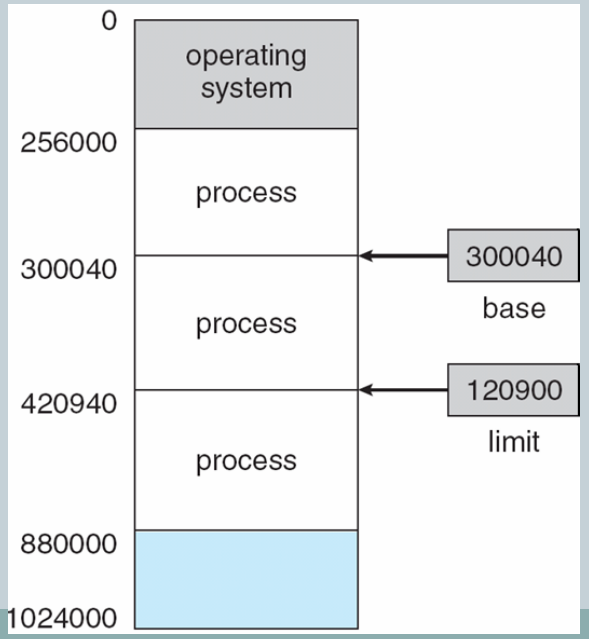
Memory Management

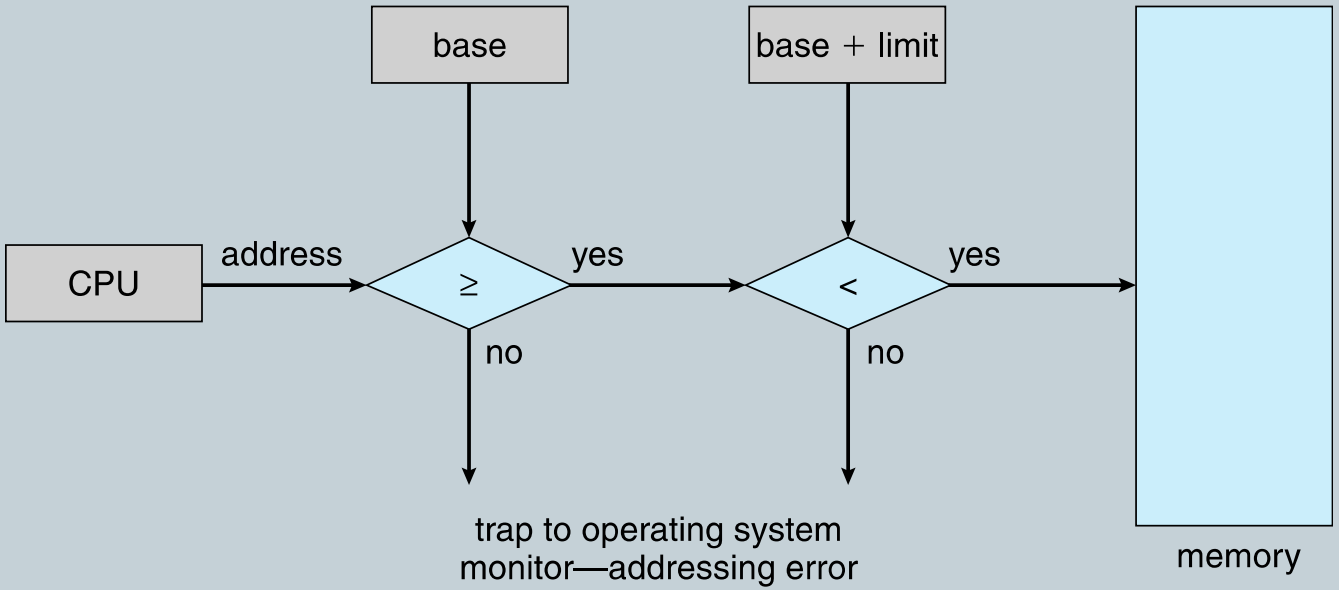


Base and Limit Registers

- A pair of **base** and **limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



Hardware Address Protection



Address Binding

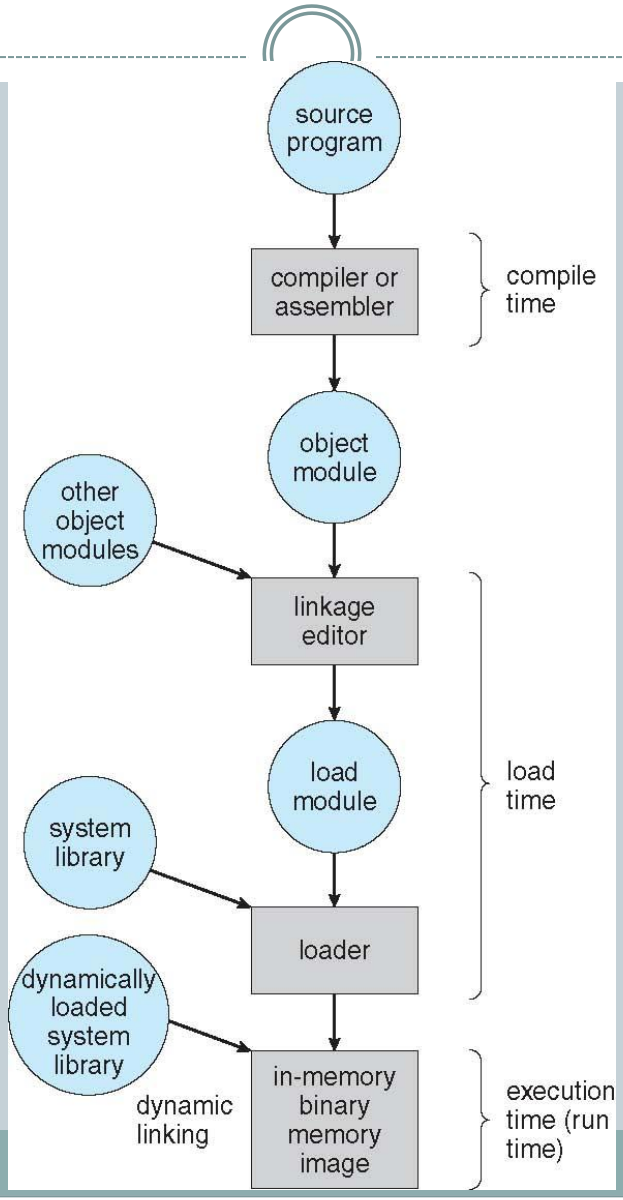


- Programs on disk
- Brought into memory to execute
- physical address of the first process is 0x0000

Binding of Instructions and Data to Memory

- Address binding happens at three different stages
  - Compile time:
  - Load time:
  - Execution time:

Multistep Processing of a User Program



Logical vs. Physical Address Space

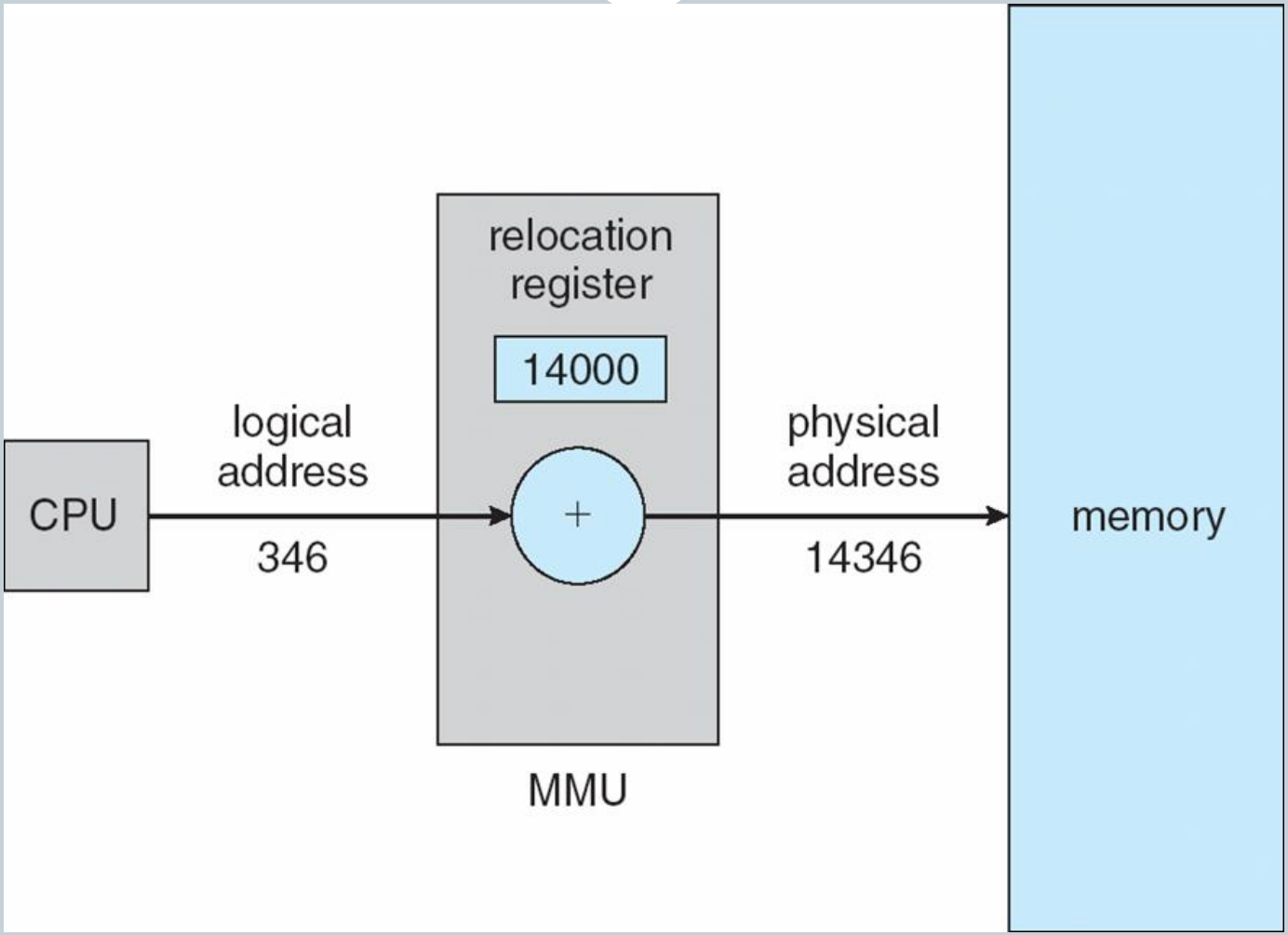


- logical address space that is bound with **physical address space**
  - **Logical address(virtual address)** – generated by the CPU
  - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time
- logical (virtual) and physical addresses differ in execution-time

Memory-Management Unit (MMU)

- At the run time: virtual address is maps in to physical address
- The user program deals with *logical* addresses (never sees the *real* physical addresses)
- Execution: Time binding occurs when reference is made to location in memory
- In MMU scheme, the value in the Relocation register (Base register) is added to every address generated by a user process at the time it is sent to memory

Dynamic relocation using a relocation register



Dynamic relocation using a relocation register

- Routine is loaded when it calls
- Unused routine is never loaded (For efficient memory-space utilization)
- All routines kept on disk in re-locatable load format
- No special support from the operating system is required
  - Implemented through program design
  - OS can help by providing libraries to implement dynamic loading

Static / Dynamic Linking



- **Static linking** : system libraries and program code combined by the loader into the binary program image

- **Dynamic linking** : link system libraries at the execution time

Dynamic Linking



- Small piece of code is called , **stub**
- stub used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks the routine is in processes' memory address.
- Dynamic linking is particularly useful for libraries (shared libraries)

Swapping

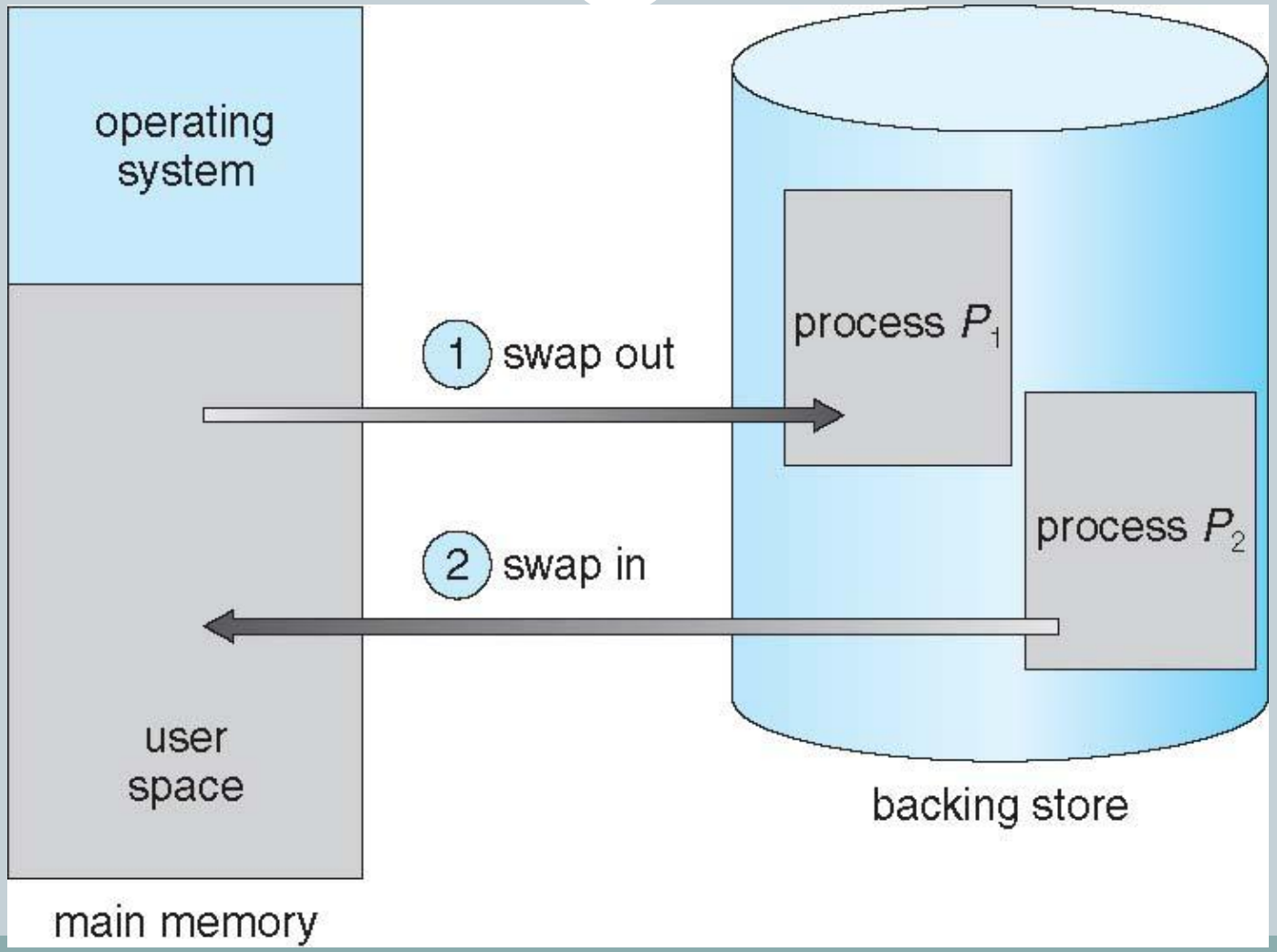


- Swapping is a mechanism that temporarily transfer the process from main memory (or move) to secondary storage (disk) and make that memory available to other processes.

Swapping

- **Backing store**:
  - fast disk and enough space to accommodate for all users
  - provide direct access to these memory images
- **Roll out, roll in**:
  - swapping mechanism is based on priority
  - lower-priority process is swapped out
  - higher-priority process can be loaded and executed
- **Swap time**: total transfer time directly proportional to the amount of memory swapped

Schematic View of Swapping



Context Switch Time including Swapping

- 100MB process swapping to hard disk with transfer rate of 50MB/sec
  - Swap out time of 2000 ms
  - Plus swap in of same sized process
  - Total context switch swapping component time of 4000ms (4 seconds)

Context Switch Time including Swapping



- Constraints on swapping
  - Pending I/O can't swap out as I/O would occur to wrong process
  - Always transfer I/O to kernel space, then to I/O device (double buffering, adds overhead)
- Standard swapping **not** used in **modern** operating systems.
- Swap only when free memory extremely low.

Swapping on Mobile Systems



- Not typically supported (Flash memory based)
  - Small amount of space
  - Limited number of write cycles
  - Poor throughput between flash memory and CPU on mobile platform

Swapping on Mobile Systems



- When free memory is low
  - iOS ***asks*** apps to voluntarily relinquish allocated memory
    - Read-only data thrown out and reloaded from flash if needed
    - Failure to free can result in termination
  - Android terminates apps if low free memory
  - Both OSes support paging

Contiguous Allocation



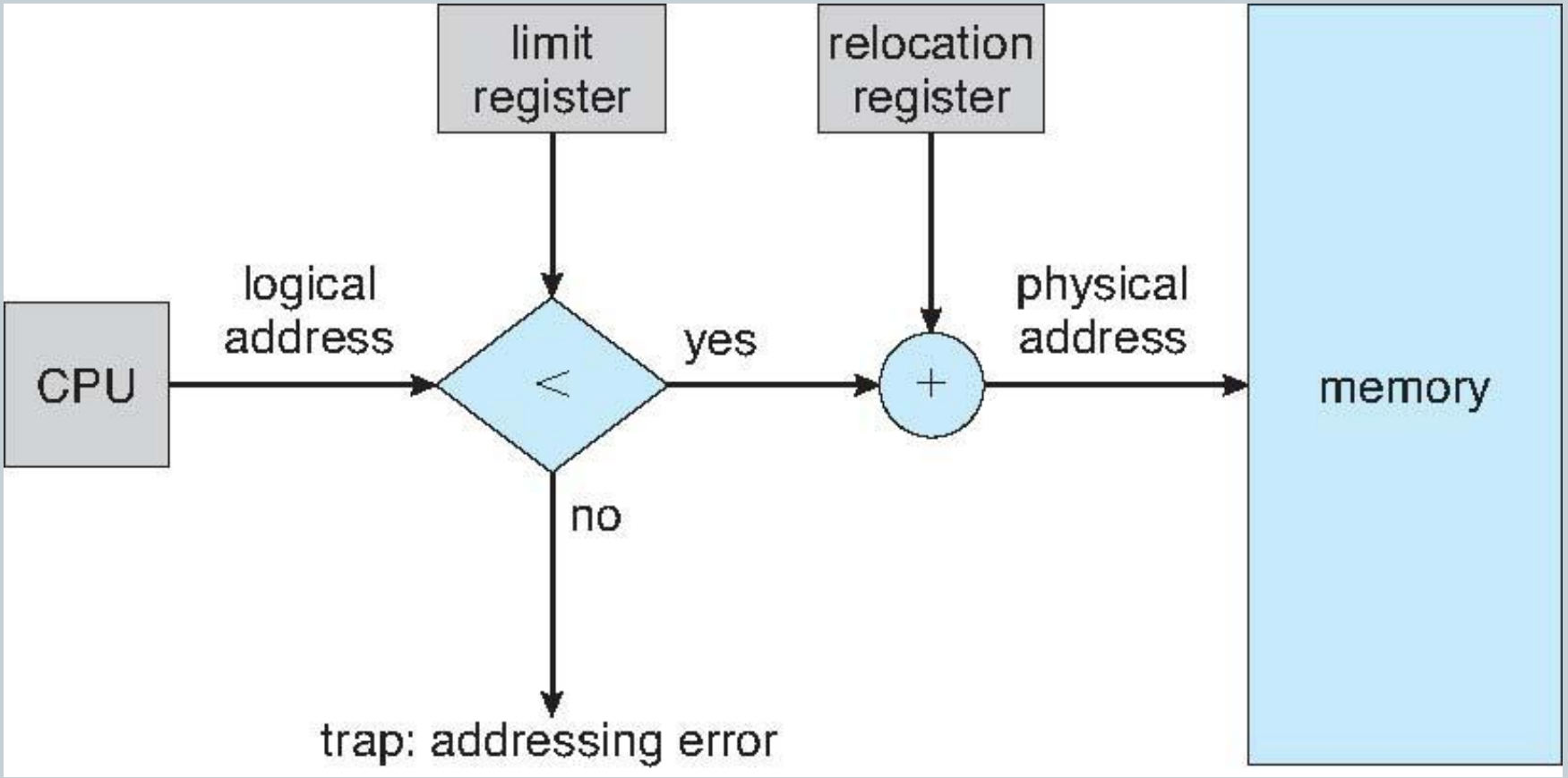
- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- Contiguous allocation is one of the early method
- Main memory usually into two **partitions**:
  - Resident operating system (held in low memory)
  - User processes in high memory
  - Each process contained in single contiguous section of memory

Contiguous Allocation



- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
  - Base register contains value of smallest physical address
  - Limit register contains range of logical addresses – each logical address must be less than the limit register
  - MMU maps logical address *dynamically*

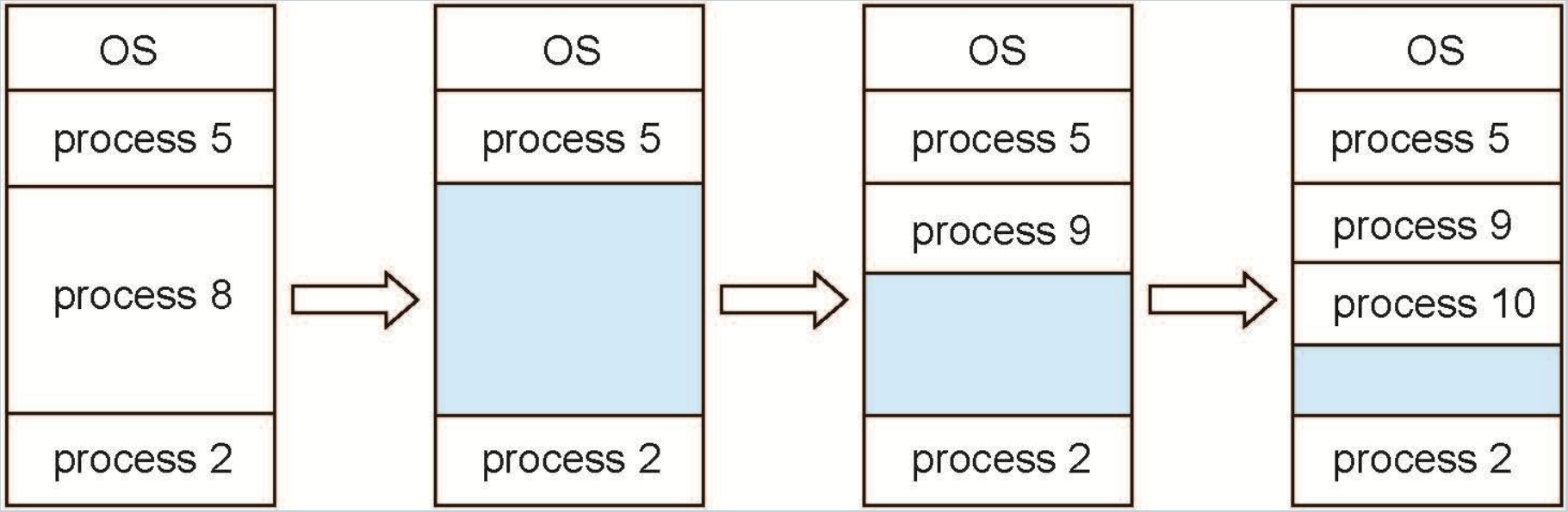
Hardware Support for Relocation and Limit Registers



Multiple-partition allocation

- Multiple-partition allocation
  - Degree of multiprogramming limited by number of partitions
  - **Variable-partition** sizes for efficiency (sized to a given process' needs)
  - **Hole** – block of available memory
  - Holes are scattered throughout memory (various size)
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Operating system maintains information about:
    - Allocated partitions
    - Free partitions (hole)

Multiple-partition allocation



Dynamic Storage-Allocation Problem



- How to satisfy a request of size n from a list of free holes?
- First-fit: Allocate the **first** hole that is big enough
- Best-fit: Allocate the **smallest** hole that is big enough
- Worst-fit: Allocate the **largest** hole

Fragmentation



- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition

Fragmentation



- Reduce external fragmentation by **compaction**
  - Shuffle memory contents to place all free memory together in one large block
  - Compaction is possible *only* if relocation is dynamic, and is done at execution time
  - I/O problem
    - Latch job in memory while it is involved in I/O
    - Do I/O only into OS buffers

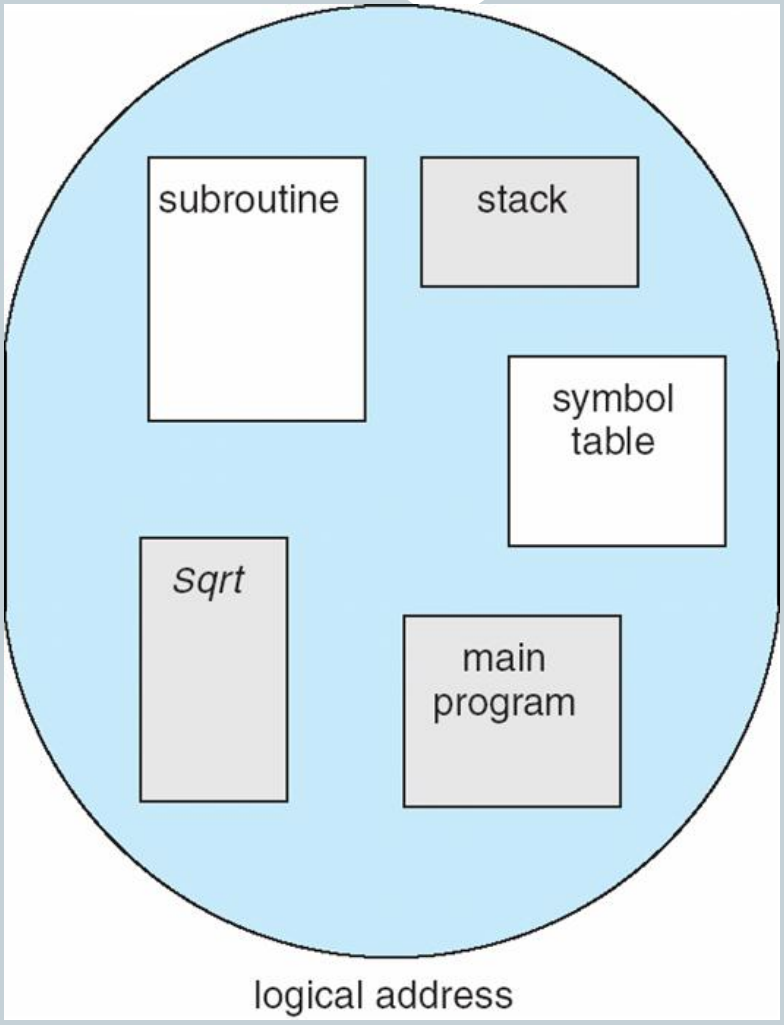
Segmentation



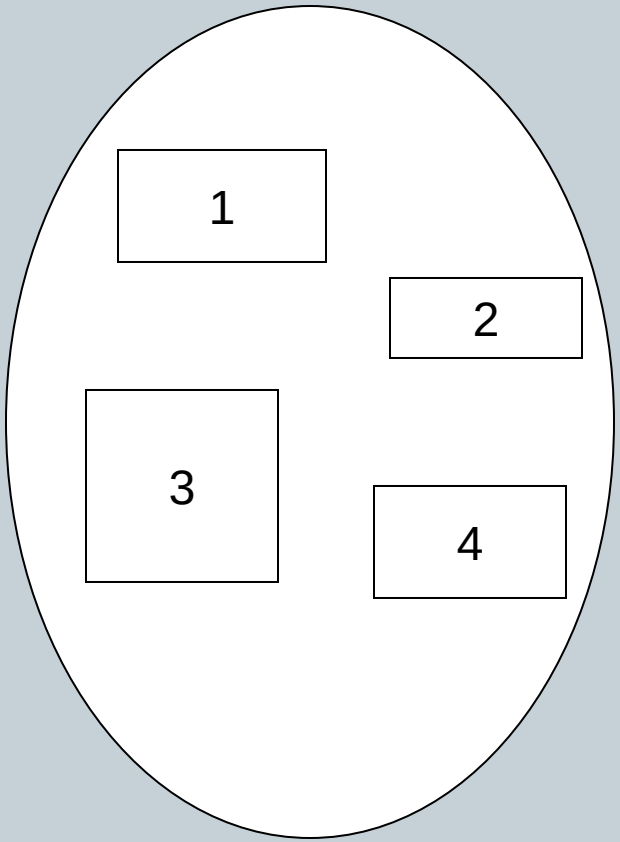
- Memory-management scheme that supports user view of memory
- A program is a collection of segments
- A segment is a logical unit

  main program, procedure , function, method, object, local variables, global variables, common block, stack, symbol table, arrays

User's View of a Program



Logical View of Segmentation



user space



physical memory space

Segmentation Architecture

- Logical address consists of a two tuple :

  <segment-number, offset>,

- **Segment table** – maps two-dimensional physical addresses; each table entry has:
  - **base** – contains the starting physical address where the segments reside in memory
  - **limit** – specifies the length of the segment

Segmentation Architecture

- **Segment-table base register (STBR) :** points to the segment table's location in memory

- **Segment-table length register (STLR) :** indicates number of segments used by a program;

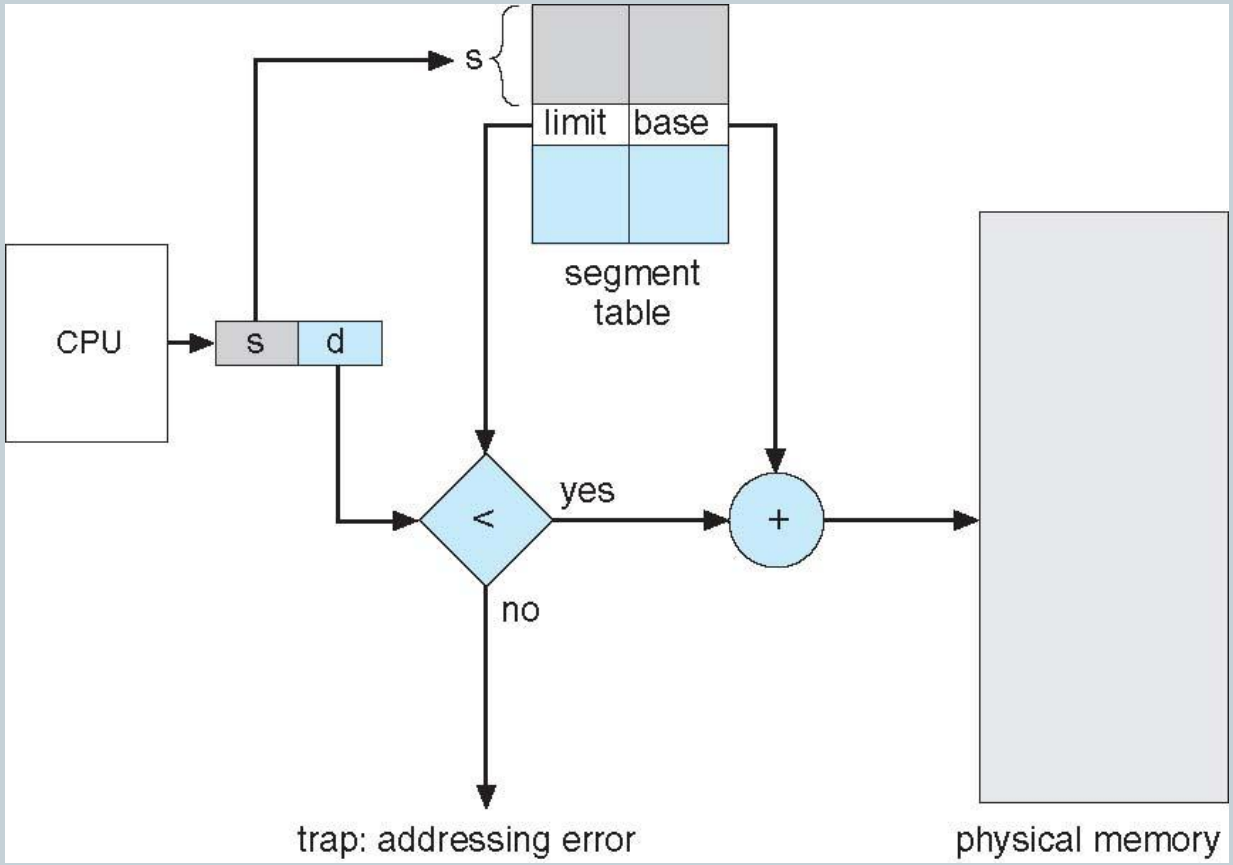
  segment number ***s*** is legal if ***s*** < **STLR**

Segmentation Architecture



- Protection
  - With each entry in segment table associate:
    - validation bit = 0 (illegal segment)
    - read/write/execute privileges
- Protection bits associated with segments; code sharing occurs at segment level

Segmentation Hardware



Paging



- Physical address space may not be contiguous
- Divide physical memory into fixed-sized blocks called **frames**
  - Size is power of 2, between 512 bytes and 16Mbytes
- Divide logical memory into blocks of same size called **pages**

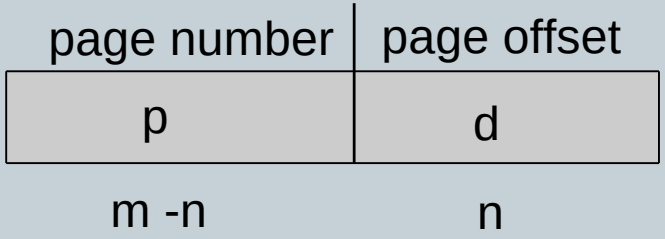
Paging



- To run a program of size ***N*** pages, need to find ***N*** free frames
- **page table** used to translate logical to physical addresses
- Backing store likewise split into pages
- Still have Internal fragmentation

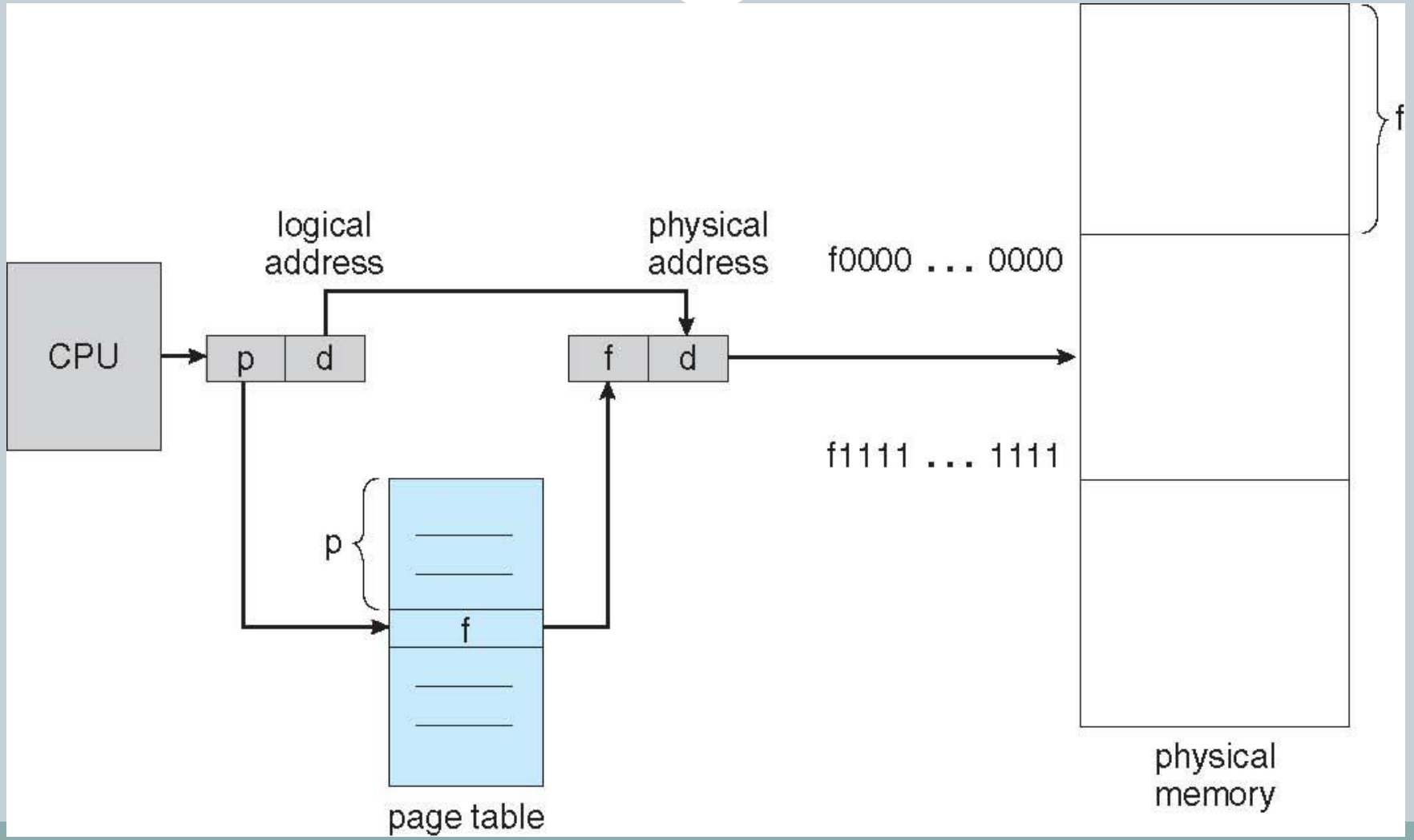
Address Translation Scheme

- Address generated by CPU is divided into:
  - **Page number** (***p***) – used as an index into a **page table** which contains base address of each page in physical memory
  - **Page offset** (***d***) – combined with base address to define the physical memory address that is sent to the memory unit

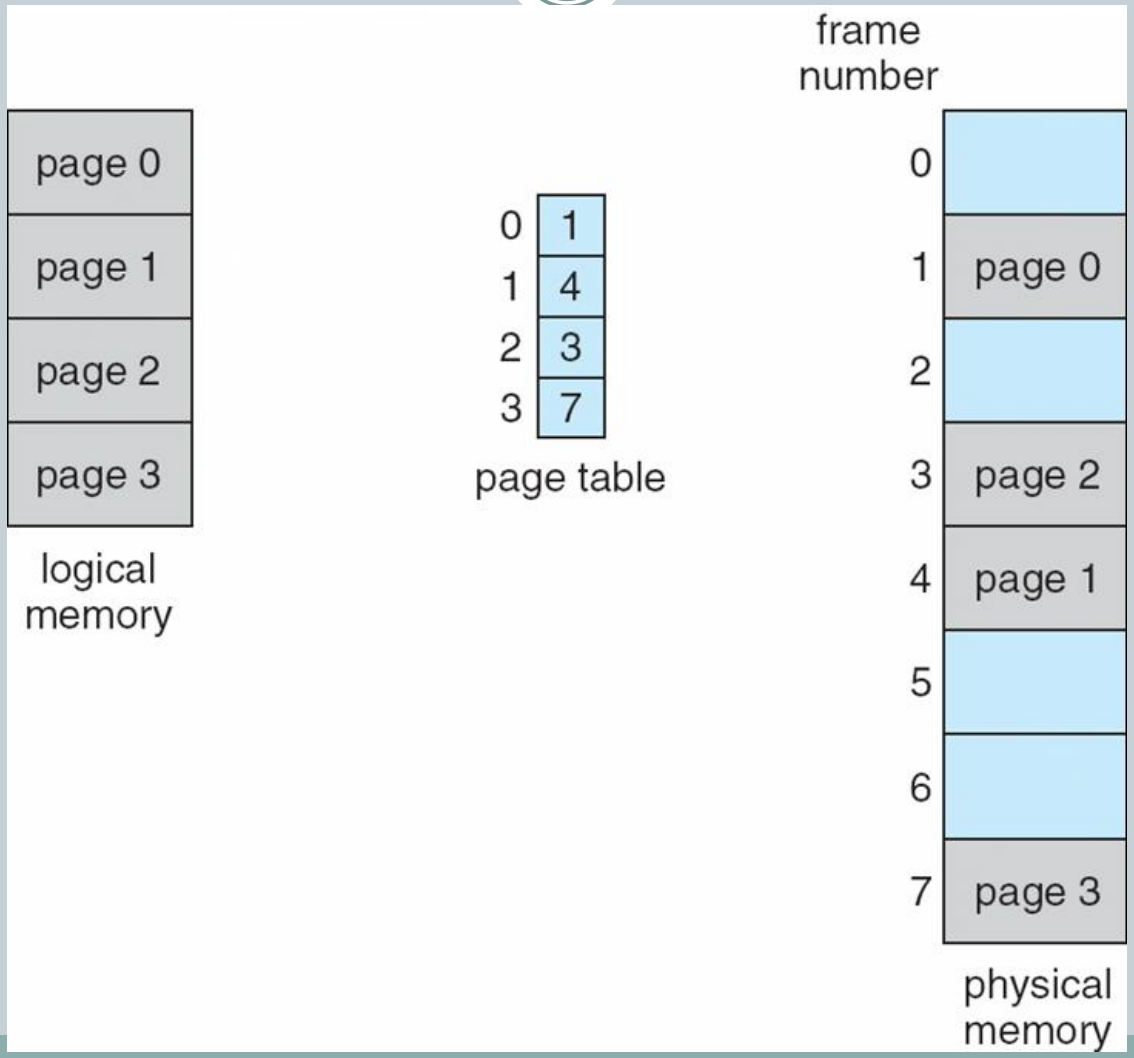


  - For given logical address space 2^m and page size 2^n

Paging Hardware

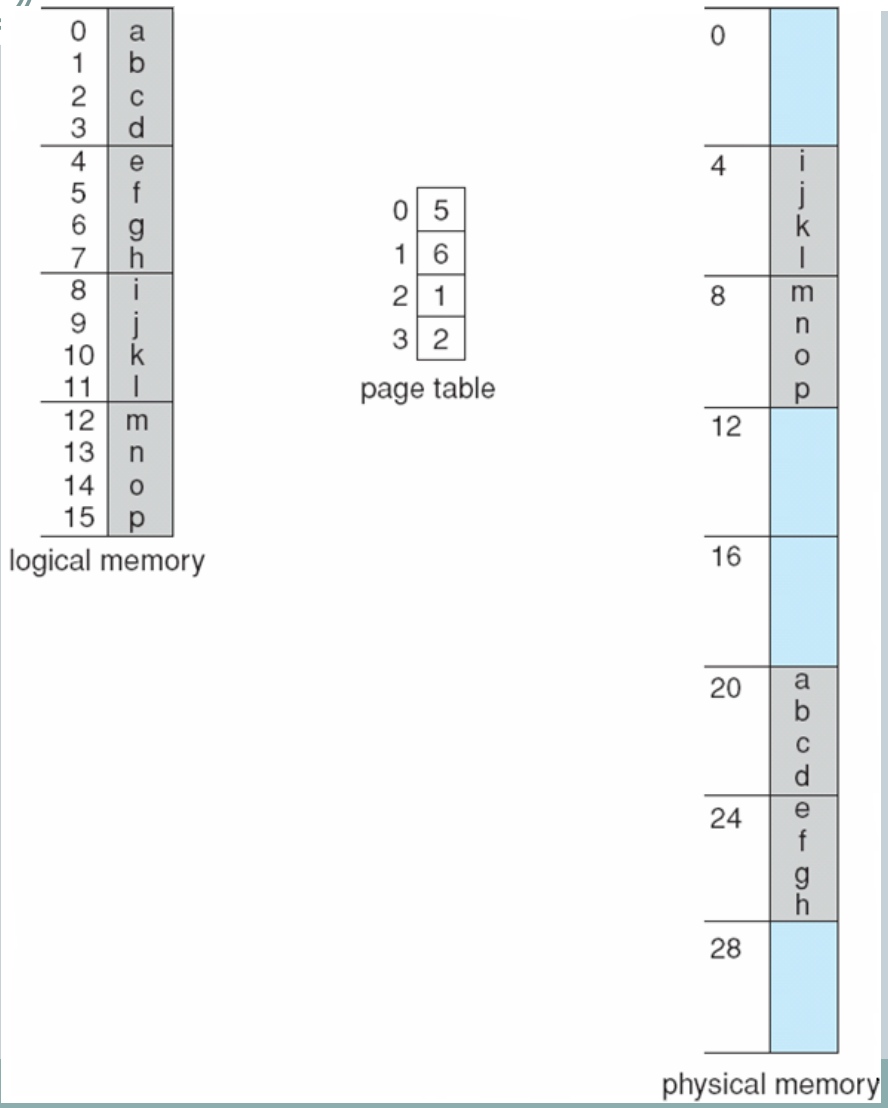


Paging Model of Logical and Physical Memory



Paging Example

- *n*=2
- *m*=4
- 32-byte memory
- 4-byte pages



Paging



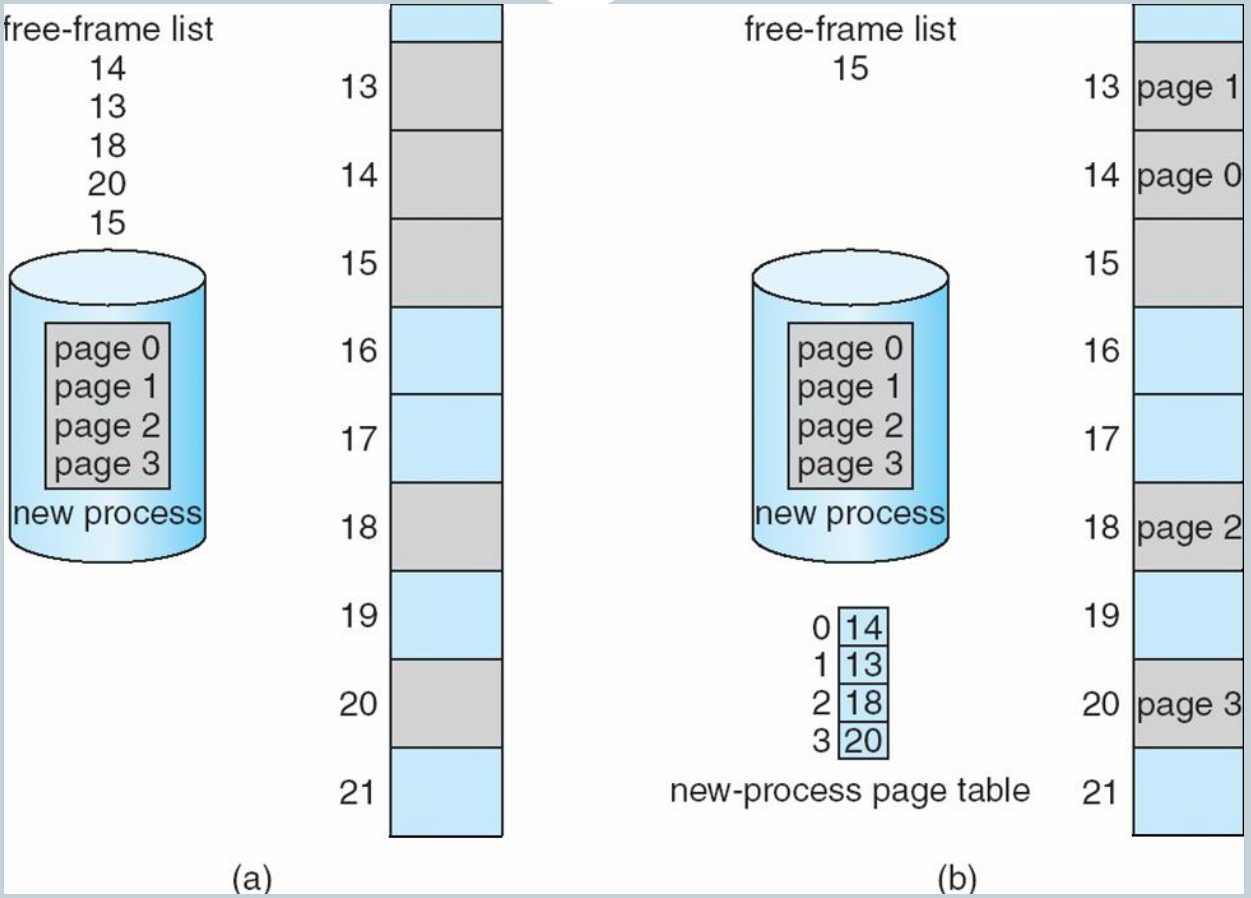
- Calculating internal fragmentation
  - Page size = 2,048 bytes
  - Process size = 72,766 bytes
  - 35 pages + 1,086 bytes
  - Internal fragmentation of 2,048 - 1,086 = 962 bytes
  - Worst case fragmentation = 1 frame – 1 byte
  - On average fragmentation = 1 / 2 frame size
  - So small frame sizes desirable?
  - But each page table entry takes memory to track
  - Page sizes growing over time

Paging



- Process view and physical memory now very different
- By implementation process can only access its own memory

Free Frames



Before allocation

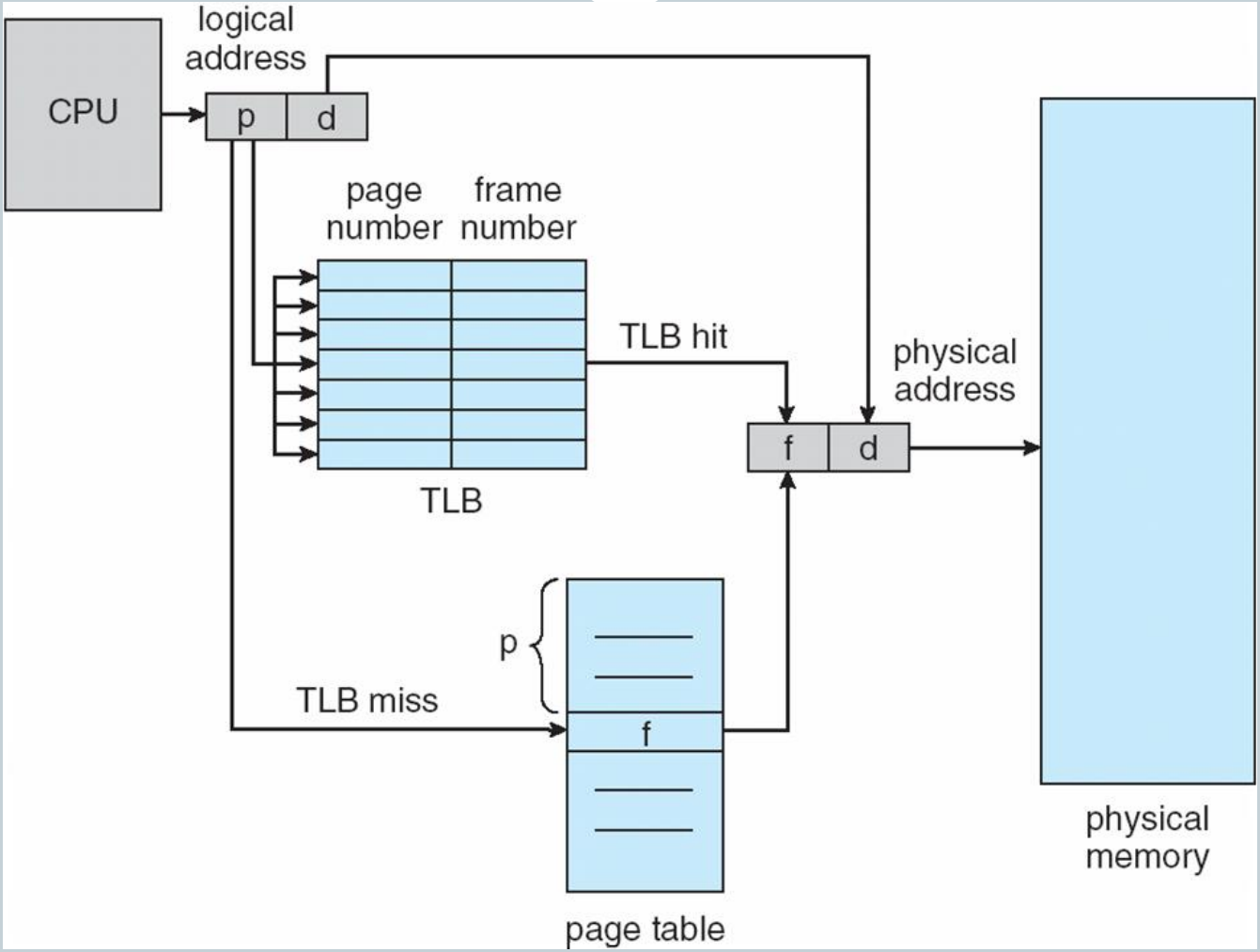
After allocation

Implementation of Page Table



- **Page-table base register** (**PTBR**): points to the page table
- **Page-table length register** (**PTLR**): indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
  - One for the page table
  - one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers** (**TLBs**)

Paging Hardware With TLB



Effective Access Time

- Associative Lookup = ε time unit
  - Can be < 10% of memory access time
- Hit ratio = α
  - Hit ratio – percentage of times that a page number is found in the associative registers
  - ratio related to number of associative registers
- Consider α = 80%, ε = 20ns for TLB search, 100ns for memory access

Effective Access Time

- **Effective Access Time (EAT)**

  = (Associative Lookup Time + Memory Cycle Time) × Hit Ratio

  +

  (Associative Lookup Time + 2×Memory Cycle Time) × (1 - Hit Ratio)

  $EAT = (1 + \varepsilon)\,\alpha + (2 + \varepsilon)(1 - \alpha)$

  $= 2 + \varepsilon - \alpha$

Effective Access Time

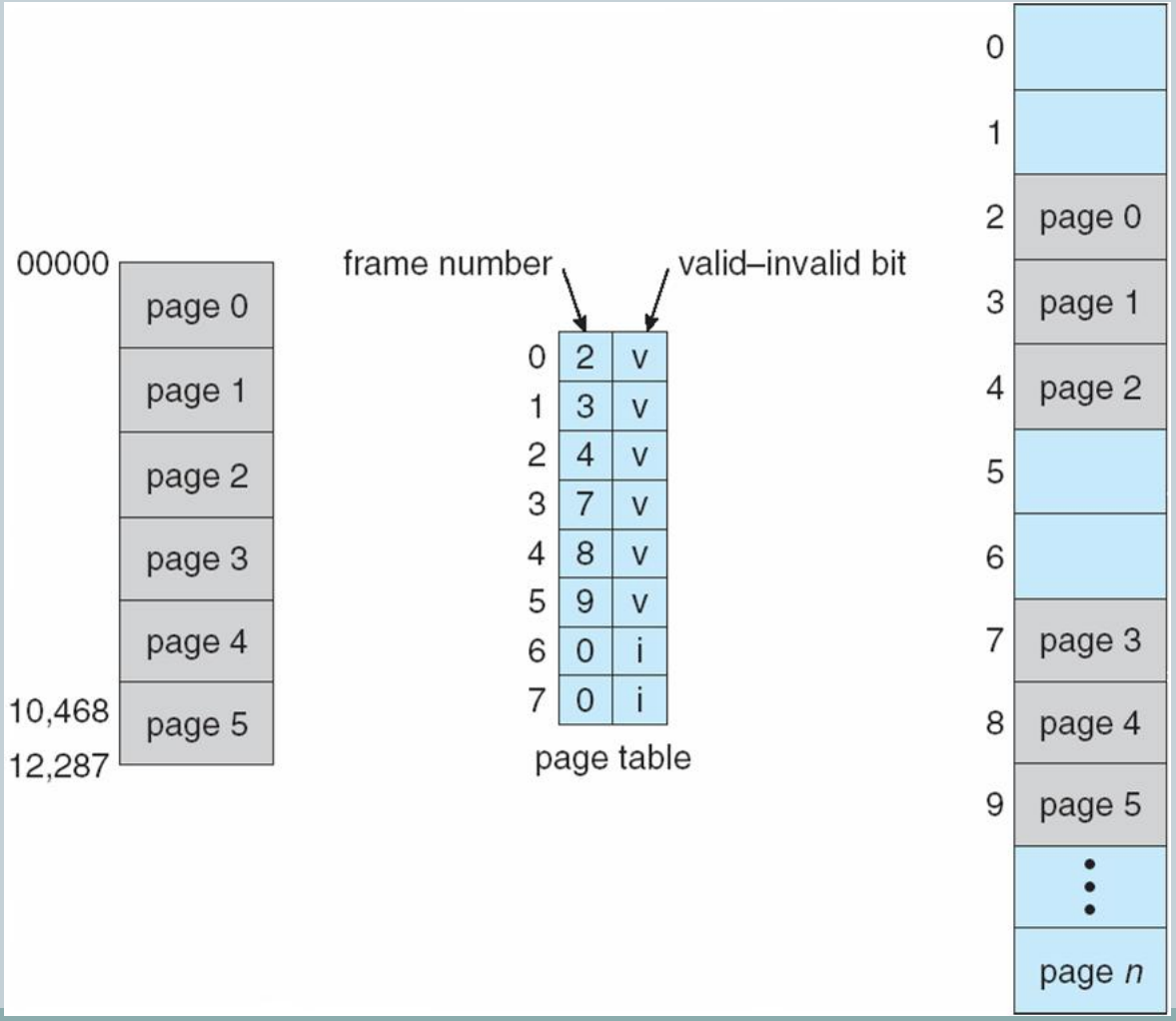


- Consider α = 80%, ε = 20ns for TLB search, 100ns for memory access
  - EAT = 0.80 x 100 + 0.20 x 200 = 120ns
- Consider more realistic hit ratio -> α = 99%, ε = 20ns for TLB search, 100ns for memory access
  - EAT = 0.99 x 100 + 0.01 x 200 = 101ns

Memory Protection

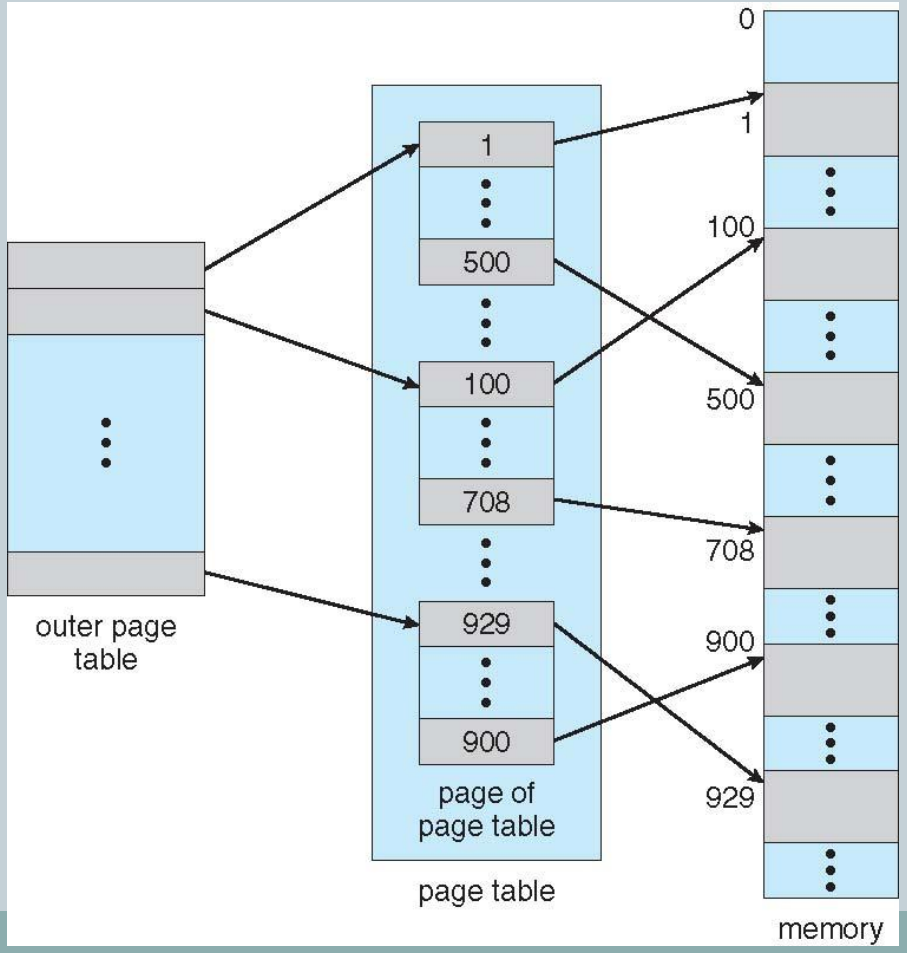
- *Valid-invalid* bit attached to each entry in the page table:
  - valid : page is in the process' logical address space, and is thus a legal page.
  - invalid :page is not in the process' logical address space.

Valid (v) or Invalid (i) Bit In A Page Table



Hierarchical Page Tables

- Two-Level Page-Table Scheme

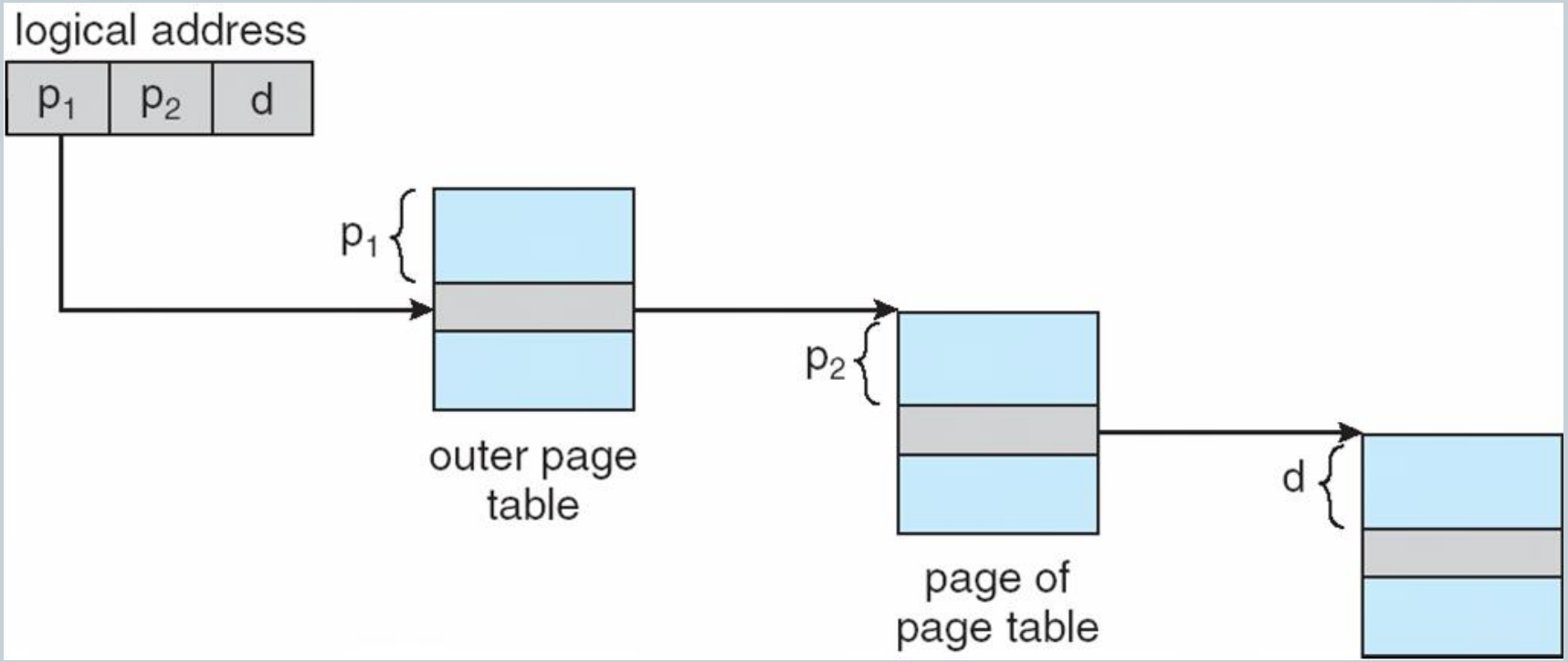


Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
  - page number :22 bits
  - page offset :10 bits
- Since the page table is paged, the page number is further divided into:
  - 12-bit page number
  - 10-bit page offset

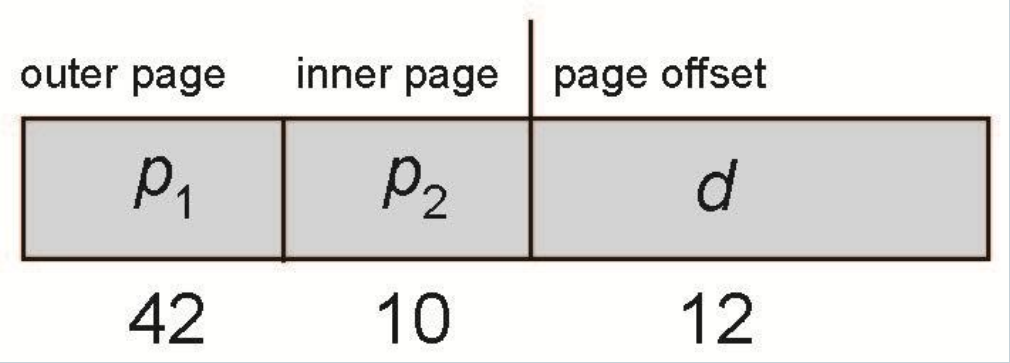
page number		page offset
p_1	p_2	d
12	10	10

Address-Translation Scheme



64-bit Logical Address Space

- Address would look like



Three-level Paging Scheme



outer page	inner page	offset
p_1	p_2	d
42	10	12

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12